

Lab 4 - Vibe Coding: Chat History

Project: 39_endless-runner (SETAPESU26) · **SRN:** PES1UG24AM039

Tool used: Claude (claude.ai) · **Chat title:** SE Vibe coding project

Shared chat link: <https://claude.ai/share/1ab778f7-cb38-4a0c-91aa-fb731cd15ce8>

Total prompts used: **3** (Task 1; Tasks 2 + 3; Task 4)

Prompt 1 — User

I'm fixing a Pygame endless runner (files attached; I'm using pygame-ce on Windows). I'll give you the README tasks in a few prompts. Rules for every answer: keep the existing structure (main.py + game/ package), don't change anything I didn't ask for, and give me COMPLETE updated files (not snippets) for every file you change, then list which files changed.

Task 1 - collision tunneling. The speed increases 0.003 px/frame forever with no cap, and collision only checks rects after the obstacle has moved. Once speed exceeds player.width + obstacle.width (55 px/frame), an obstacle can jump from fully right of the player to fully left in one frame, so no hit registers and it even gets scored. Fix both: (1) add a MAX_SPEED cap in GameEngine with a playable value; (2) make collision swept by testing the rect covering each obstacle's previous and current x position against the player rect. Also add a debug key F1 that sets speed to 80 and bypasses the cap, so I can test that standing still always dies. Dont ask questions to test me

Response 1 — Claude

Only game/game_engine.py changes. main.py already forwards every event to engine.handle_event, so F1 needs no change there.

game/game_engine.py

```
import math
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    MAX_SPEED = 20
    DEBUG_SPEED = 80

    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)

        self.speed = 6
        self.speed_increase_per_frame = 0.003
        self.speed_cap_enabled = True

        self.spawn_interval = 70 # frames between obstacle spawns
        self.spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False
```

```

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
        self.player.jump()
    elif event.type == pygame.KEYDOWN and event.key == pygame.K_F1:
        self.speed = self.DEBUG_SPEED
        self.speed_cap_enabled = False

def handle_input(self):
    # Reserved for continuously-held-key input; this runner only
    # needs an edge-triggered jump, handled in handle_event.
    pass

def update(self):
    if self.game_over:
        return

    self.speed += self.speed_increase_per_frame
    if self.speed_cap_enabled:
        self.speed = min(self.speed, self.MAX_SPEED)
    self.player.update()

    self._spawn_timer += 1
    if self._spawn_timer >= self.spawn_interval:
        self._spawn_timer = 0
        self.obstacles.append(Obstacle(self.width, self.ground_y, self.speed))

    previous_x = [obstacle.x for obstacle in self.obstacles]
    for obstacle in self.obstacles:
        obstacle.move()
        obstacle.speed = self.speed

    player_rect = self.player.rect()
    for obstacle, prev_x in zip(self.obstacles, previous_x):
        if self._swept_rect(obstacle, prev_x).colliderect(player_rect):
            self.game_over = True
            return

    for obstacle in self.obstacles:
        if not obstacle.scored and obstacle.x + obstacle.width < self.player.x:
            obstacle.scored = True
            self.score += 1

    self.obstacles = [o for o in self.obstacles if not o.off_screen()]

    self.distance += self.speed

def _swept_rect(self, obstacle, prev_x):
    left = math.floor(min(prev_x, obstacle.x))
    right = math.ceil(max(prev_x, obstacle.x) + obstacle.width)
    return pygame.Rect(left, obstacle.y, right - left, obstacle.height)

def render(self, screen):
    pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width, self.ground_y), 4)

    pygame.draw.rect(screen, WHITE, self.player.rect())
    for obstacle in self.obstacles:
        pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

    score_text = self.font.render(f"Score: {self.score}", True, (0, 0, 0))
    screen.blit(score_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        # NOTE: no proper game-over screen yet - see Task 2 in the README.
        print("Game over! Final score:", self.score)
        self._game_over_logged = True

```

Files changed: game/game_engine.py only.

Beyond what you asked for, I made three small edits. I removed the two NOTE comments about Task 1, because they described the bug and would now be wrong. I made 80 a named constant `DEBUG_SPEED` (the reason is below). I also added `import math`, which is used for the floor and ceiling in the swept rect.

How the swept test works

Before the move loop runs, each obstacle's x is saved. After the move, the engine builds one rect that stretches from

the obstacle's left edge at its new x to its right edge at its old x. That rect covers every x position the obstacle passed through during the frame, and it is tested against the player.

Here is a concrete case at speed 75. The player occupies x from 80 to 110.

- **Old check:** the obstacle goes from x=125, covering 125 to 150 (fully right of the player), to x=50, covering 50 to 75 (fully left). Neither rect overlaps 80 to 110, so no hit registers. On the same frame, $50+25 < 80$ is true, so the obstacle even gets scored.
- **Swept check:** the swept rect is x from 50 to 150. It overlaps 80 to 110, so the hit registers.

The collision check returns before the scoring loop, so a tunnelling obstacle can no longer be scored.

The swept rects are laid out end to end from one frame to the next, with no gaps between them. That means standing still at any speed must hit, which is exactly what your F1 test checks.

Why `MAX_SPEED = 20`

- **Jump window:** with gravity 0.8 and jump strength -15, the player's bottom stays above the 40 px obstacle for about 31 frames of each roughly 37-frame jump. At 20 px/frame, the obstacle needs about 3 frames ($55/20$) to pass the player's 55 px overlap zone. That leaves a timing window of about 28 frames, roughly 0.47 s. This is forgiving.
- **Reaction time:** the obstacle crosses the roughly 720 px approach in 36 frames, which is 0.6 s. That is fast but still human-playable. At 30 px/frame it drops to 0.4 s, which gets brutal.
- **Spacing:** at the cap, obstacles are $70 \times 20 = 1400$ px apart, so only one is on screen at a time.
- **Ramp:** getting from 6 to 20 at 0.003 per frame takes about 4,700 frames, around 78 s. The difficulty ramp still has room to build.
- **Safety margin:** 20 is far below 55, so even the old check wouldn't tunnel at the cap. The swept test is defense in depth, and it is what makes F1 safe.

Two things to know

1. At speed 80, the old code also always dies. A freshly spawned obstacle's positions are 800 - 80k, which gives 720, 640, ..., 160, 80. It lands exactly on x=80, directly over the player. The old code hits there too. So F1=80 confirms the new code dies, but it doesn't prove the sweep is what caught it.

Speeds that genuinely skip the hitbox include 75 (... , 125, 50) and 85 (... , 120, 35). Faster obstacles tunnel only when every landing spot misses the 55 px danger zone. That's why I made 80 a constant: set `DEBUG_SPEED = 75` to see the swept check do real work.

For your "before" video, the original code has no F1. Reaching speed 55 would take about 16,300 frames, roughly 4.5 minutes, and even then tunnelling depends on how the positions line up. The quickest demo is to temporarily set `self.speed = 75` in the untouched original, record it, then revert.

2. The sweep can be one frame strict when landing. The swept x-range is tested against the player's end-of-frame y. Suppose the player is falling and the obstacle's trailing edge passes under them in the same frame. That can count as a hit even if, in continuous time, the obstacle had already left before the player dropped low enough.

At speed 20, this affects at most 20 px behind the obstacle during one 16 ms frame, which is imperceptible. It's the standard trade-off of your one-axis approach. A fully exact version would interpolate the player's y as well and intersect the two overlap time intervals. It isn't needed at this cap.

Prompt 2 — User

Tasks 2 and 3 together. Game Over: currently collision just freezes the game and `render()` prints to the console. Add states (PLAYING, GAME_OVER) in `GameEngine`. On `GAME_OVER`, stop gameplay updates, ignore jump keys, and draw a semi-transparent overlay showing "GAME OVER", the final score, and the key options. Remove the console print and the `_game_over_logged` hack. Replay: on the Game Over screen, keys 1/2/3 restart on Easy/Medium/Hard and Q/Esc quits cleanly through `main.py`'s loop (no `sys.exit` inside the engine). Keep presets in one `DIFFICULTIES` dict (Easy speed 5 spawn 90, Medium 6/70, Hard 8/55). Restart via a `reset(difficulty)` method that resets ALL run state (player position, velocity, on_ground, obstacles, timers, score, distance, speed). Show the current difficulty on screen. Ignore key presses for 0.5 s after dying so a mashed key can't skip the screen. Give complete updated files.

Response 2 — Claude

Two files change. `game/game_engine.py` gets the state machine, presets, `reset()` and the overlay. `main.py` gets a two-line quit check, because the engine only raises a flag and `main.py`'s loop does the actual exiting.

`game/game_engine.py`

```
import math
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

PLAYING = "PLAYING"
GAME_OVER = "GAME_OVER"

DIFFICULTIES = {
    "Easy": {"key": pygame.K_1, "speed": 5, "spawn_interval": 90},
    "Medium": {"key": pygame.K_2, "speed": 6, "spawn_interval": 70},
    "Hard": {"key": pygame.K_3, "speed": 8, "spawn_interval": 55},
}

class GameEngine:
    MAX_SPEED = 20
    DEBUG_SPEED = 80
    GAME_OVER_INPUT_DELAY_MS = 500

    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.speed_increase_per_frame = 0.003

        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 22)

        self.overlay = pygame.Surface((width, height), pygame.SRCALPHA)
        self.overlay.fill((0, 0, 0, 160))

        self.quit_requested = False
        self.reset("Medium")

    def reset(self, difficulty):
        preset = DIFFICULTIES[difficulty]
        self.difficulty = difficulty

        self.player = Player(80, self.ground_y)

        self.speed = preset["speed"]
        self.speed_cap_enabled = True

        self.spawn_interval = preset["spawn_interval"] # frames between obstacle spawns
        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0

        self.state = PLAYING
        self.game_over_time = 0

    def handle_event(self, event):
        if event.type != pygame.KEYDOWN:
            return

        if self.state == PLAYING:
            if event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
                self.player.jump()
            elif event.key == pygame.K_F1:
                self.speed = self.DEBUG_SPEED
```

```

        self.speed_cap_enabled = False

    elif self.state == GAME_OVER:
        if pygame.time.get_ticks() - self.game_over_time < self.GAME_OVER_INPUT_DELAY_MS:
            return
        if event.key in (pygame.K_q, pygame.K_ESCAPE):
            self.quit_requested = True
            return
        for name, preset in DIFFICULTIES.items():
            if event.key == preset["key"]:
                self.reset(name)
                return

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
        if self.state != PLAYING:
            return

        self.speed += self.speed_increase_per_frame
        if self.speed_cap_enabled:
            self.speed = min(self.speed, self.MAX_SPEED)
        self.player.update()

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y, self.speed))

        previous_x = [obstacle.x for obstacle in self.obstacles]
        for obstacle in self.obstacles:
            obstacle.move()
            obstacle.speed = self.speed

        player_rect = self.player.rect()
        for obstacle, prev_x in zip(self.obstacles, previous_x):
            if self._swept_rect(obstacle, prev_x).colliderect(player_rect):
                self.state = GAME_OVER
                self.game_over_time = pygame.time.get_ticks()
                return

        for obstacle in self.obstacles:
            if not obstacle.scored and obstacle.x + obstacle.width < self.player.x:
                obstacle.scored = True
                self.score += 1

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]

        self.distance += self.speed

    def _swept_rect(self, obstacle, prev_x):
        left = math.floor(min(prev_x, obstacle.x))
        right = math.ceil(max(prev_x, obstacle.x) + obstacle.width)
        return pygame.Rect(left, obstacle.y, right - left, obstacle.height)

    def _blit_centered(self, screen, font, text, center_y, color=WHITE):
        surface = font.render(text, True, color)
        screen.blit(surface, surface.get_rect(center=(self.width // 2, center_y)))

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width, self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

        score_text = self.font.render(f"Score: {self.score}", True, (0, 0, 0))
        screen.blit(score_text, (10, 10))

        difficulty_text = self.font.render(f"Difficulty: {self.difficulty}", True, (0, 0, 0))
        screen.blit(difficulty_text, difficulty_text.get_rect(topright=(self.width - 10, 10)))

        if self.state == GAME_OVER:
            screen.blit(self.overlay, (0, 0))

```

```

self.blit_centered(screen, self.title_font, "GAME OVER", 120)
self.blit_centered(screen, self.font, f"Final score: {self.score} ({self.difficulty})", 190)
options = " ".join(
    f"{pygame.key.name(preset['key'])} - {name}" for name, preset in DIFFICULTIES.items()
)
self.blit_centered(screen, self.small_font, f"Play again: {options}", 250)
self.blit_centered(screen, self.small_font, "Q / Esc - Quit", 285)

```

main.py

```

import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 800, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Endless Runner - Pygame Version")

# Colors
SKY = (200, 220, 240)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(SKY)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)
            if engine.quit_requested:
                running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()

```

Files changed: game/game_engine.py, main.py. (player.py and obstacle.py are unchanged.)

Design notes

State machine. self.game_over (a bool) is replaced by self.state. Each method checks the state it cares about:

Method	PLAYING	GAME_OVER
handle_event	jump keys, F1	1/2/3, Q/Esc (after 0.5 s)
update	full simulation	returns immediately
render	world + HUD	world + HUD + overlay

The frozen scene still draws under the overlay, so you can see what killed you. The console print and _game_over_logged are gone.

reset() is the single source of run state. __init__ calls reset("Medium"), so the first run and every replay go through the same code path. If a future task adds a new run variable, you add it in one place and can't forget to reset it.

The player is reset by constructing a fresh `Player(80, self.ground_y)`, which restores `x`, `y`, `vy` and `on_ground` without touching `player.py`. `reset()` also sets `speed_cap_enabled = True`, so a previous F1 debug run can't leak into the next game. Medium (6/70) matches the original hardcoded values, so startup behaves exactly as before.

One dict, three uses. Each `DIFFICULTIES` entry holds its own key, so the restart handler loops over the dict and the options line on the overlay is generated from it too. If you add a "Brutal" preset with `pygame.K_4`, it works and appears on screen with no other edits.

Input delay. The delay is measured in wall-clock milliseconds (`pygame.time.get_ticks()`), not frames, so it is a true 0.5 s even if FPS dips. Key presses during that window are dropped, not queued. Also, pygame doesn't auto-repeat `KEYDOWN` by default, so holding Space through the death never counts as a new press. Only a fresh press after 0.5 s does anything.

Clean quit. The engine only sets `quit_requested`. `main.py` turns that into `running = False`, finishes the current frame, and exits through its existing `pygame.quit()`. That's the same path as closing the window. Esc during gameplay does nothing; you only asked for it on the Game Over screen.

Possible additions I didn't make:

- Numpad 1/2/3 (`K_KP1-K_KP3`) are not accepted, since you specified keys 1/2/3.
- The options text is visible during the 0.5 s lockout even though the keys don't respond yet. If that feels unresponsive in testing, the options could be hidden until the delay expires.

Prompt 3 — User

Task 4 – sound. Add three short sound effects: jump, scoring (clearing an obstacle), and game over. Generate the tones in code using `pygame.mixer.Sound` from a byte buffer (no audio files, no numpy) in a new file `game/sounds.py`. Call `pygame.mixer.pre_init` before `pygame.init` in `main.py`. If the audio device fails, the game must still run silently. Play the game-over sound exactly once at the moment of collision, and the jump sound only when a jump actually starts (not mid-air presses). Give complete updated files.

Response 3 — Claude

Four files change: one is new, three are updated.

`game/sounds.py` (new)

```
import math
from array import array
import pygame

class Sounds:
    VOLUME = 0.25
    FADE_SECONDS = 0.005

    def __init__(self):
        self.enabled = False
        self._sounds = {}
        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init()
            frequency, size, channels = pygame.mixer.get_init()
            if size != -16:
                return
            self._rate = frequency
            self._channels = channels
            self._sounds = {
                "jump": self._make([(400, 800, 0.12)]),
                "score": self._make([(880, 880, 0.06), (1320, 1320, 0.08)]),
                "game_over": self._make([(440, 110, 0.6)]),
            }
            self.enabled = True
        except (pygame.error, NotImplementedError):
            self.enabled = False
            self._sounds = {}

    def _make(self, segments):
```

```

        samples = array("h")
        amplitude = self.VOLUME * 32767
        fade = max(1, int(self._rate * self.FADE_SECONDS))
        phase = 0.0
        for start_hz, end_hz, duration in segments:
            n = int(self._rate * duration)
            for i in range(n):
                hz = start_hz + (end_hz - start_hz) * (i / n)
                phase += 2 * math.pi * hz / self._rate
                value = amplitude if math.sin(phase) >= 0 else -amplitude
                envelope = min(1.0, i / fade, (n - i) / fade)
                sample = int(value * envelope)
                for _ in range(self._channels):
                    samples.append(sample)
        return pygame.mixer.Sound(buffer=samples.tobytes())

    def play(self, name):
        if self.enabled:
            self._sounds[name].play()

```

game/player.py

```

import pygame

class Player:
    def __init__(self, x, ground_y, width=30, height=40):
        self.x = x
        self.ground_y = ground_y
        self.width = width
        self.height = height
        self.y = ground_y - height
        self.vy = 0
        self.gravity = 0.8
        self.jump_strength = -15
        self.on_ground = True

    def jump(self):
        if self.on_ground:
            self.vy = self.jump_strength
            self.on_ground = False
            return True
        return False

    def update(self):
        self.vy += self.gravity
        self.y += self.vy

        ground_level = self.ground_y - self.height
        if self.y >= ground_level:
            self.y = ground_level
            self.vy = 0
            self.on_ground = True

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)

```

game/game_engine.py

```

import math
import pygame
from .player import Player
from .obstacle import Obstacle
from .sounds import Sounds

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

PLAYING = "PLAYING"
GAME_OVER = "GAME_OVER"

DIFFICULTIES = {

```

```

"Easy": {"key": pygame.K_1, "speed": 5, "spawn_interval": 90},
"Medium": {"key": pygame.K_2, "speed": 6, "spawn_interval": 70},
"Hard": {"key": pygame.K_3, "speed": 8, "spawn_interval": 55},
}

class GameEngine:
    MAX_SPEED = 20
    DEBUG_SPEED = 80
    GAME_OVER_INPUT_DELAY_MS = 500

    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.speed_increase_per_frame = 0.003

        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 22)

        self.overlay = pygame.Surface((width, height), pygame.SRCALPHA)
        self.overlay.fill((0, 0, 0, 160))

        self.sounds = Sounds()

        self.quit_requested = False
        self.reset("Medium")

    def reset(self, difficulty):
        preset = DIFFICULTIES[difficulty]
        self.difficulty = difficulty

        self.player = Player(80, self.ground_y)

        self.speed = preset["speed"]
        self.speed_cap_enabled = True

        self.spawn_interval = preset["spawn_interval"] # frames between obstacle spawns
        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0

        self.state = PLAYING
        self.game_over_time = 0

    def handle_event(self, event):
        if event.type != pygame.KEYDOWN:
            return

        if self.state == PLAYING:
            if event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
                if self.player.jump():
                    self.sounds.play("jump")
            elif event.key == pygame.K_F1:
                self.speed = self.DEBUG_SPEED
                self.speed_cap_enabled = False

        elif self.state == GAME_OVER:
            if pygame.time.get_ticks() - self.game_over_time < self.GAME_OVER_INPUT_DELAY_MS:
                return
            if event.key in (pygame.K_q, pygame.K_ESCAPE):
                self.quit_requested = True
                return
            for name, preset in DIFFICULTIES.items():
                if event.key == preset["key"]:
                    self.reset(name)
                    return

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):

```

```

if self.state != PLAYING:
    return

self.speed += self.speed_increase_per_frame
if self.speed_cap_enabled:
    self.speed = min(self.speed, self.MAX_SPEED)
self.player.update()

self._spawn_timer += 1
if self._spawn_timer >= self.spawn_interval:
    self._spawn_timer = 0
    self.obstacles.append(Obstacle(self.width, self.ground_y, self.speed))

previous_x = [obstacle.x for obstacle in self.obstacles]
for obstacle in self.obstacles:
    obstacle.move()
    obstacle.speed = self.speed

player_rect = self.player.rect()
for obstacle, prev_x in zip(self.obstacles, previous_x):
    if self._swept_rect(obstacle, prev_x).collidect(player_rect):
        self.state = GAME_OVER
        self.game_over_time = pygame.time.get_ticks()
        self.sounds.play("game_over")
        return

for obstacle in self.obstacles:
    if not obstacle.scored and obstacle.x + obstacle.width < self.player.x:
        obstacle.scored = True
        self.score += 1
        self.sounds.play("score")

self.obstacles = [o for o in self.obstacles if not o.off_screen()]

self.distance += self.speed

def _swept_rect(self, obstacle, prev_x):
    left = math.floor(min(prev_x, obstacle.x))
    right = math.ceil(max(prev_x, obstacle.x) + obstacle.width)
    return pygame.Rect(left, obstacle.y, right - left, obstacle.height)

def _blit_centered(self, screen, font, text, center_y, color=WHITE):
    surface = font.render(text, True, color)
    screen.blit(surface, surface.get_rect(center=(self.width // 2, center_y)))

def render(self, screen):
    pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width, self.ground_y), 4)

    pygame.draw.rect(screen, WHITE, self.player.rect())
    for obstacle in self.obstacles:
        pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

    score_text = self.font.render(f"Score: {self.score}", True, (0, 0, 0))
    screen.blit(score_text, (10, 10))

    difficulty_text = self.font.render(f"Difficulty: {self.difficulty}", True, (0, 0, 0))
    screen.blit(difficulty_text, difficulty_text.get_rect(topright=(self.width - 10, 10)))

    if self.state == GAME_OVER:
        screen.blit(self.overlay, (0, 0))
        self._blit_centered(screen, self.title_font, "GAME OVER", 120)
        self._blit_centered(screen, self.font, f"Final score: {self.score} ({self.difficulty})", 190)
        options = " ".join(
            f"{pygame.key.name(preset['key'])} - {name}" for name, preset in DIFFICULTIES.items()
        )
        self._blit_centered(screen, self.small_font, f"Play again: {options}", 250)
        self._blit_centered(screen, self.small_font, "Q / Esc - Quit", 285)

```

main.py

```

import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
try:

```

```

pygame.mixer.pre_init(44100, -16, 1, 512, allowedchanges=0)
except (pygame.error, NotImplementedError):
    pass
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 800, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Endless Runner - Pygame Version")

# Colors
SKY = (200, 220, 240)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(SKY)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)
            if engine.quit_requested:
                running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()

```

Files changed: game/sounds.py (new), game/player.py, game/game_engine.py, main.py. (obstacle.py is unchanged.)

How the sound generation works

The byte buffer is raw PCM. `pre_init(44100, -16, 1, 512)` asks for 44,100 samples per second, signed 16-bit integers, and mono. A sound is then just a list of integers in the range -32768 to 32767 , one per $1/44100$ s.

For example, the 0.12 s jump is $0.12 \times 44100 = 5,292$ samples. At 2 bytes each, that's 10,584 bytes. `array("h")` stores signed 16-bit ints in native byte order, and -16 means "signed 16-bit, system byte order", so `.tobytes()` is exactly what the mixer expects. That's how this works without numpy.

Why `allowedchanges=0`. Without it, SDL may open the device in whatever format the hardware prefers, such as stereo or 48 kHz. Your hand-built buffer would then play at the wrong pitch or speed. Setting `allowedchanges=0` tells SDL to always give pygame the requested format and convert internally.

`Sounds` still reads the real values from `get_init()` as a second line of defence. It uses the actual sample rate, duplicates samples if the mixer is stereo, and goes silent rather than playing noise if the sample size isn't 16-bit.

Sweeps use phase accumulation. The obvious formula $\sin(2\pi \cdot f(t) \cdot t)$ has a classic bug. The pitch you hear is the rate of change of the phase, which is $f(t) + f'(t) \cdot t$, not $f(t)$. For the jump's 400→800 Hz sweep, $f' = 400/0.12$ Hz/s. At $t = 0.12$ s you'd hear $800 + 400 = 1200$ Hz, not 800.

Accumulating phase $\pm 2\pi \cdot \text{hz}/\text{rate}$ each sample advances the phase by exactly the current frequency, so the sweep ends at 800 Hz as intended. This also keeps the phase continuous across the score sound's two segments (880 Hz, then 1320 Hz), so there's no click at the join.

Envelope against clicks. A square wave that starts or stops at full amplitude produces an audible pop. `envelope = min(1, i/fade, (n-i)/fade)` ramps volume up over the first 5 ms and down over the last 5 ms. That's 220 samples at 44.1 kHz.

The three sounds:

- Jump: a rising 400→800 Hz chirp (0.12 s).
- Score: a two-note blip, 880→1320 Hz, a rising fifth (0.14 s).
- Game over: a falling 440→110 Hz slide, two octaves down (0.6 s).

All are generated once at startup, about 36k samples in total, which takes a few tens of milliseconds. `play()` is then just a mixer call.

Behaviour guarantees

- **Silent fallback.** `pygame.init()` never raises when a subsystem fails; it just counts it in its return value. So with no audio device, the mixer simply isn't initialised. `Sounds` then tries `mixer.init()`, which raises `pygame.error`. That error is caught, and `enabled = False` makes every `play()` a no-op. `NotImplementedError` covers `pygame` builds without the mixer module. The `try` around `pre_init` in `main.py` is only a guard for the same case.
- **Jump sound only on a real takeoff.** `Player.jump()` now returns `True` only when it actually launches from the ground. Presses in mid-air return `False` and play nothing. That's why `player.py` changed: the player is the one object that knows whether a jump happened, so the engine asks it rather than duplicating the `on_ground` check.
- **Game-over sound exactly once.** It plays on the line that switches the state to `GAME_OVER`, which is immediately followed by `return`. After that, `update()` exits on its first line every frame, so the collision code can't run again until `reset()`.
- **Score sound once per obstacle.** It is tied to the same `obstacle.scored` flag that stops an obstacle from being scored twice.

One thing I left alone: if you restart right after the 0.5 s lockout, the last ~0.1 s of the 0.6 s game-over slide plays over the start of the new run. If you'd rather cut it off, adding `pygame.mixer.stop()` in `reset()` would do it, guarded by `self.sounds.enabled`.